ST STEVAL-STWINBX1

Wideband MEMS vibration node

MCU	STM32U585AI, M33 @ 160 MHz
Vibration	IIS3DWB, **6.3 kHz flat**, ±16 g
Acquisition	8192 @ 26.667 kHz (307 ms)
RAM used	148 kB of 786 kB
Radio	BLE 5.2, 300 B frame
Battery life	**1.6 y as shipped, 15.2 y re-celled**

Branch `hw/stwinbx1-stm32u5`. Detail: `hardware/stwinbx1/README.md`, `docs/hardware/stwinbx1.md`.

Never flashed. The shared DSP is verified against the Python reference (1 113 C checks) and the application loop runs on the host, but no board has been on a bench. Timing and current figures are datasheet arithmetic.

`board_stwinbx1.c` — the IIS3DWB driver, DMA, clocks, BLE and sleep modes — is the part that needs hardware.

Why this part: bandwidth, not noise

	IIS3DWB	ADXL355	ISM330BX
Flat BW	**6.3 kHz**	1 kHz	0.9 kHz
Noise $\mu g/\sqrt{\text{Hz}}$	75	**25**	70
Full scale	±16 g	±8 g	±16 g
Resolution	16-bit	**20-bit**	16-bit
Current	1.1 mA	**0.2 mA**	0.55 mA

The ADXL355 is quieter and finer. The ESP32-S3 branch uses it for exactly that reason.

For bearing diagnosis, bandwidth wins.

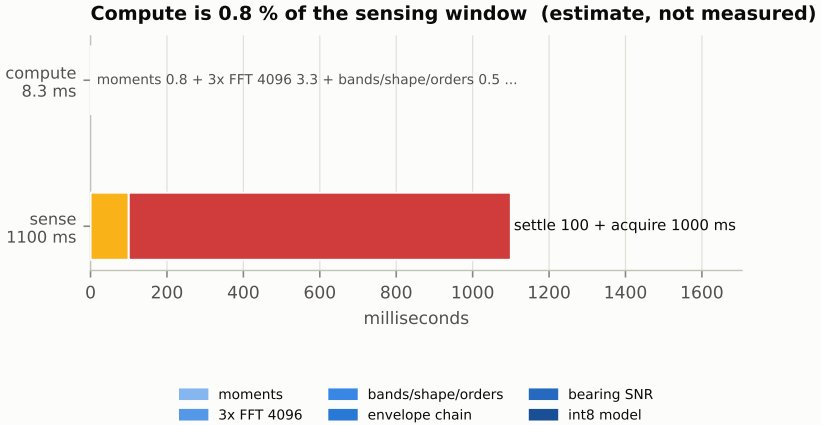
A spalled raceway produces impacts at BPFO (≈106 Hz here). The diagnostic information is *not* at 106 Hz — it is in the **modulation at 106 Hz of a structural resonance at 2–20 kHz**.

A sensor that rolls off at 1 kHz sees a rougher machine. It cannot see *which* bearing element is damaged, no matter how low its noise floor.

And the IIS3DWB has exactly **one** output rate. No anti-alias filter to configure, no bandwidth/ODR ratio to get wrong, no mode where the part silently delivers less bandwidth than expected — which on configurable parts is a common and nearly undetectable bug, because the data just looks smoother.

So what: A magnet mount rolls off above ∼2 kHz and throws all of this away. Mount type is recorded per device for that reason.

The compute is not the problem

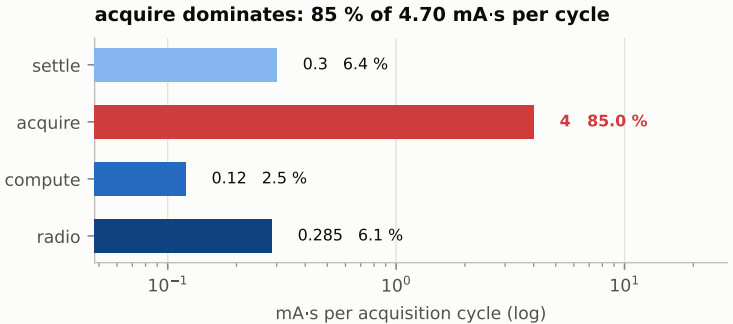


8.6 ms of DSP against a 407 ms sensing window: **2.1 %**. At one acquisition per hour that is a 2.4 ppm compute duty cycle.

So there is no case on compute grounds for the NPU- or Helium-bearing alternatives in this lineup. See the Alif branch for what an NPU actually buys, which is a *bigger model*, not a faster

Where the margin went instead. 786 kB of SRAM is luxurious for this workload — the pipeline uses 148 kB — so this kit can afford a 16384-sample acquisition (1.63 Hz envelope resolution) that the constrained kits cannot. It runs 8192 by default only for comparability across branches.

Where the energy actually goes



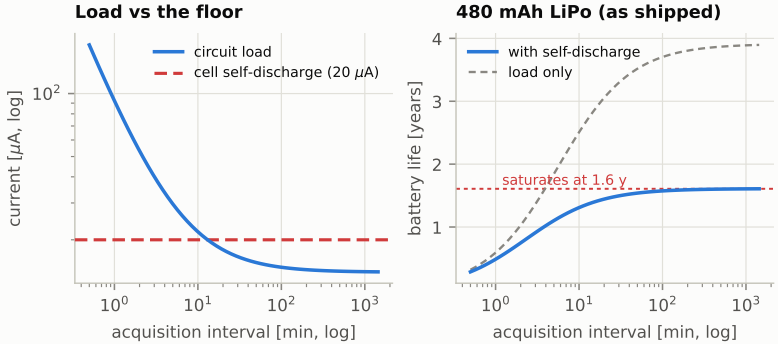
The acquisition window dominates: 85 %. Not the radio (6 %), not the DSP (2.5 %).

Most published vibration-node designs optimise the radio and the FFT. On this kit those are the two smallest terms. The levers that matter are a shorter record — which costs envelope resolution — and the IIS3DWB's 1.1 mA operating current, which is fixed by the part.

A 330× error this figure caught.
`APP_SENSOR_KEEPALIVE_THRESHOLD_S` was 90 s, on the stated reasoning that staying powered beats re-settling above that interval. Settling is 0.30 mA·s once; keeping the sensor alive is 1.1 mA *continuously*. The crossover is **0.27 seconds**.

At 90 s the node would have burned 99 mA·s per cycle instead of 0.30 — battery life in weeks. Caught by a test that recomputes the crossover from the energy model rather than trusting the constant.

The cell, not the firmware, is the binding constraint



The shipped 480 mAh LiPo self-discharges at 3 %/month — **20 μA**, more than the circuit's 15.3 μA average draw. Battery life *saturates at 1.6 years* regardless of interval; beyond ∼15 minutes nothing the firmware does can improve it. The dashed curve is the circuit alone.

Swap the chemistry:

```
power_budget.py --kit stwinbx1 \
  --battery-mah 2400 --self-discharge-pct-month 0.08
```

$\Rightarrow$ **15.2 years**, with no firmware change.

$LiSOCl_2$ passivates after long storage and has a pulse-current limit that matters on the cellular kit — but for a fit-and-forget vibration node it is the correct chemistry.

The open metrological problem: ±3 % sample rate

The IIS3DWB's rate comes from an internal oscillator specified at ±3 %, and this board offers no way to measure the true value: the FIFO carries no timestamps at 26.667 kHz.

At the 3 % extreme a 106.3 Hz BPFO line is computed as if it were at 103.1 Hz. The matcher's ±2 % tolerance absorbs that — just. By the **fifth harmonic** the error is 16 Hz against a ±10.6 Hz window, so the top harmonics fall outside and `n_matched` under-reports exactly when the evidence is strongest.

Three fixes, one chosen

1. Reference tone at commissioning. Planned. The self-test actuator produces a known mechanical deflection; drive it from an MCU timer and measure where the tone lands. Gives the true rate to 0.02 %. One extra second at commissioning, stored per device.

2. Widen the tolerance to ±4 %. Cheap and wrong: doubles the chance of matching a neighbouring family, which is what turns an inner-race diagnosis into an outer-race one.

3. Track the 2× line-frequency component. Free and as accurate as the grid — but assumes the asset is mains-driven, false for anything on a VFD.

So what: Until fix 1 ships, the firmware reports the nominal rate and the limitation is written down rather than hidden.

Two unused parts are more interesting than anything left to tune

IMP23ABSU microphone, 80 kHz

Complementary, not redundant. Needs *no mechanical coupling*, so it sidesteps the mounting problem that dominates accelerometer practice — a microphone hears a bearing through air where an accelerometer needs a stud into the housing.

80 kHz also reaches the low-ultrasonic band where lubrication distress appears earliest. The DCASE lineage in the research branch is built on exactly this signal, with a maintained MIT-licensed toolkit (`TakuyaFujimura/dcase-asd-toolkit`).

The analogue output goes to the U5's ADC: firmware work, not hardware.

ISM330BX MLC/ISPU as a wake-up filter

85 % of this kit's energy is the acquisition window. A decision tree running *inside* the accelerometer on low-rate data, waking the MCU only when the vibration signature changes, attacks the dominant term directly. At long intervals the saving could be an order of magnitude.

The cost is a second model to train, validate and version — and a new failure mode where the wake-up filter's false-negative rate silently becomes the system's detection limit.

Worth doing. Worth doing carefully.

Multi-axis is the obvious third option and is not obviously worth it: three axes triple the dominant energy term, and bearing faults rarely need an axial channel. Misalignment and looseness do.